

Sigourney Reipsch 104871

In dieser Übung habe ich zwei Android-Anwendungen entwickelt, um die Konzepte der persistenten Datenspeicherung relationaler Datenbanken sowie die Auswertung von Hardware Sensoren zu zeigen.

Für die Aufgaben 1 und 2 habe ich eine Einkaufslisten-App programmiert. Diese basiert auf der Android Room-Datenbank Bibliothek und implementiert ein Entity-Relationship-Modell (ERM) mit drei verknüpften Tabellen (Laden, Kategorie, Produkt). Der Nutzer kann über eine Benutzeroberfläche mit Drop-down-Menüs neue Produkte anlegen, die mitsamt ihren Zuordnungen persistent gespeichert und in einer Liste ausgegeben werden. Einge kaufte Produkte lassen sich per Wischgeste wieder aus der Datenbank entfernen. Ich gebe für Aufgabe 1 keinen expliziten Code ab, da ich mich strikt an das Tutorial gehalten habe. Ich denke, alles relevante finden Sie im Code der 2 Aufgabe.

Für die Aufgaben 3 und 4 habe ich eine digitale Wasserwaagen-App entwickelt. Diese liest kontinuierlich die Rohdaten des geräteinternen Beschleunigungssensors aus, berechnet die Neigungswinkel aller drei Achsen und gibt diese numerisch aus. Zusätzlich habe ich eine grafische Oberfläche implementiert, die eine Wasserwaagen-Blase simuliert. Die Bewegungen der Blase wurden durch einen Puffer geglättet, um ein Zittern der Anzeige zu verhindern.

App 1 Einkaufsliste:

Die Architektur der App folgt dem MVVM-Muster. Das Datenmodell besteht aus den drei Entitätsklassen *Laden*, *Kategorie* und *Produkt*, die über Fremdschlüssel (*kategorieIdZuordnung*, *ladenIdZuordnung*) verknüpft sind. Das Interface *EinkaufDao* definiert die SQL-Befehle. Eine Besonderheit ist hierbei der *@Query*-Befehl *getProdukteMitDetails()*: Dieser nutzt einen *INNER JOIN*, um die IDs der Produkte direkt beim Auslesen mit den Klartext-Namen aus den Laden- und Kategorie-Tabellen zu einer neuen Hilfsklasse zusammenzuführen. Das *EinkaufRepository* verwaltet die Hintergrund-Threads für Schreibzugriffe, während Lesezugriffe als *LiveData* über das *EinkaufViewModel* an die *MainActivity* gereicht werden. Die UI nutzt einen *ListAdapter*, um die *RecyclerView* effizient zu aktualisieren. Gelöscht wird über den *ItemTouchHelper*, der Wischgesten registriert und die ID des weggewischten Elements an die Löschroutine der Datenbank übergibt.

App 2 Wasserwaage:

Die App implementiert das *SensorEventListener*-Interface. Beim Aufruf von *onResume* wird der Sensor über den *SensorManager* mit einer moderaten Verzögerung registriert. Die Methode *onSensorChanged* fängt die einströmenden X-, Y- und Z-Rohdaten ab. Über den Arcustangens werden die Vektoren in Grad-Winkel konvertiert. Für die flüssige grafische Darstellung durchlaufen die X- und Y-Winkel die Hilfsklasse *Smoother*, die einen gleitenden Durchschnitt über ein Array der letzten 10 Werte bildet. Die Grafik selbst wird in der Methode *drawControl* erzeugt. Um Speicherüberläufe zu verhindern, werden *Bitmap*, *Canvas* und *Paint*-Objekte einmalig in der *onCreate*-Methode initialisiert. In der Zeichnungsschleife wird

das alte Bild per *PorterDuff.Mode.CLEAR* gelöscht, das Raster neu gezeichnet und die Position der Wasserwaagen-Blase anhand der geglätteten Winkel dynamisch verschoben.

Codeübernahmen:

- **Fehlerbehebung (Aufgabe 1 und 4):** Für das Debugging von Gradle-Fehlern (SDK-Versionskonflikte) und Emulator Konflikten wurde das KI-Modell **Gemini (Version Juni 2026, Google)** genutzt.
- **Grafische Wasserwaage (Aufgabe 4):** Die Methodik zur Berechnung der X- und Y-Winkel, die Methode `drawControl(double rot_x, double rot_y)` zum Zeichnen der Canvas-Elemente sowie die Schnittstelle `IInstruments` und die Hilfsklasse `Smoother` wurden direkt aus den bereitgestellten Vorlesungsunterlagen übernommen:
 - *Anmerkung:* Der Code wurde von Gemini leicht modifiziert, um Abstürze zu verhindern (Vorbefüllung des `Smoother-Arrays` mit 0-Werten).

Schwierigkeiten und Lösungen:

Bei der Bearbeitung dieser Übung habe ich tiefgreifende Einblicke in die Verknüpfung lokaler SQLite-Datenbanken und in die direkte Verarbeitung von Hardware-Sensorik unter Android gewonnen. Besonders lehrreich war das Mapping von Relationen in objektorientierten Kotlin-Code und das asynchrone Aktualisieren der UI mithilfe von `LiveData` und `Views`.

Während der Entwicklung traten einige **Schwierigkeiten** auf, die gelöst werden mussten:

1. **Datenbank-Updates:** Bei der Erweiterung des Formulars um statische Drop-down-Werte zeigte sich, dass Room den *onCreate*-Block der Datenbank nur beim allerersten Start ausführt. Neue Einträge (wie zusätzliche Läden) wurden nicht sichtbar, bis die App deinstalliert und die Datenbank neu initialisiert wurde.
2. **Gradle & SDK-Konflikte:** Die Integration moderner AndroidX-Bibliotheken (Version 1.19.0) führte zu Fehlern beim Kompilieren, da das *compileSdk* auf Version 37 angehoben werden musste. Zudem gab es einen Abbruch beim Linking der XML-Ressourcen ("Adaptive Icons require SDK 26"), welcher durch das Anheben des *minSdk*-Werts von 24 auf 26 in der *build.gradle.kts* behoben werden konnte.
3. **Out-of-Memory-Crashes:** Der ursprüngliche Code der Wasserwaage generierte bei jedem Sensor-Tick ein komplett neues Bitmap-Objekt im Speicher. Dies führte schnell zu Abstürzen der App auf dem Emulator. Die Lösung bestand darin, das Bitmap und den Canvas nur ein einziges Mal global in der *onCreate*-Methode zu initialisieren und das bestehende Bild vor jedem neuen Zeichenvorgang transparent zu löschen. Außerdem musste die `Smoother`-Klasse beim Start mit Nullen initialisiert werden, um Exceptions bei sofortigen Sensor-Ausschlägen abzufangen.

1.1 Ursprüngliche Eingabeoberfläche der grundlegenden Tutorial-Anwendung, bevor diese zur Einkaufsliste umgebaut wurde.

1.2 Darstellung der gespeicherten Einträge in der initialen Tutorial-App. Die Wörter werden über eine RecyclerView als einfache, alphabetisch geordnete Liste ausgegeben.

2.1 Hauptansicht der entwickelten Einkaufslisten-App. Die Listeneinträge zeigen neben dem Produktnamen auch direkt das zugeordnete Geschäft und die Kategorie an.

2.2 Eingabeformular zum Hinzufügen neuer Produkte. Das Layout wurde um zwei Spinner erweitert, um die relationalen Daten für das ER-Modell zu erfassen.

2.3 Geöffnetes Drop-down-Menü zur Auswahl des Geschäfts. Die angezeigten Optionen entsprechen den beim Start initialisierten Einträgen der Room-Datenbank.

2.4 Geöffnetes Drop-down-Menü zur Zuweisung einer Produktkategorie, um die zweite 1:n-Beziehung beim Speichern des Produkts korrekt zu hinterlegen.

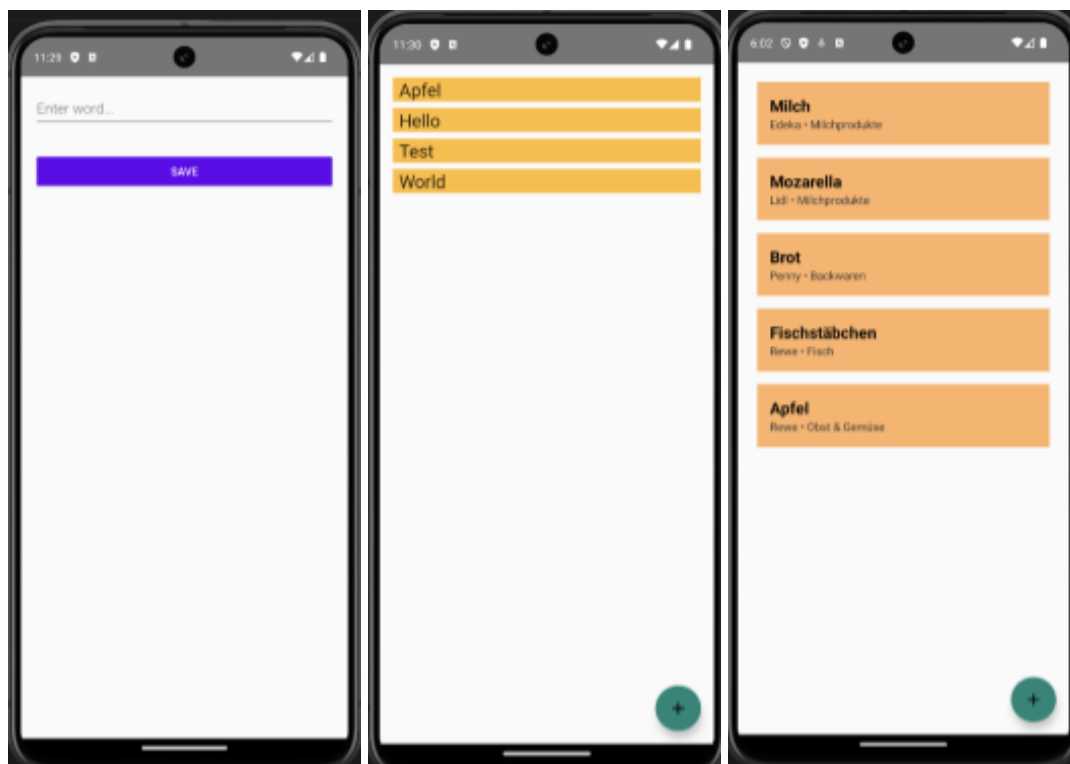
2.5 Entity-Relationship-Modell (Chen-Notation) der Einkaufslisten-App. Dargestellt sind die drei Entitäten Laden, Kategorie und Produkt sowie deren 1:n-Beziehungen.

3.1 Benutzeroberfläche der Neigungswinkel-App

3.2 Simulation der Geräte Rotation über die "Extended Controls" im Android Emulator.

4.1 Erweiterte Benutzeroberfläche der App als digitale Wasserwaage.

4.2 Test der grafischen Wasserwaage im Emulator. Die Position der Blase passt sich der Lageänderung dynamisch an.



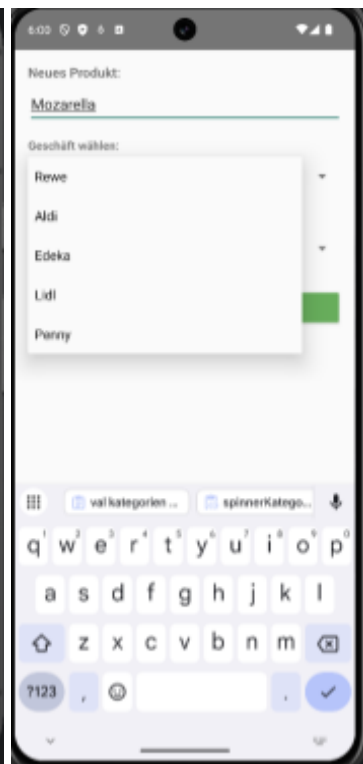
1.1

1.2

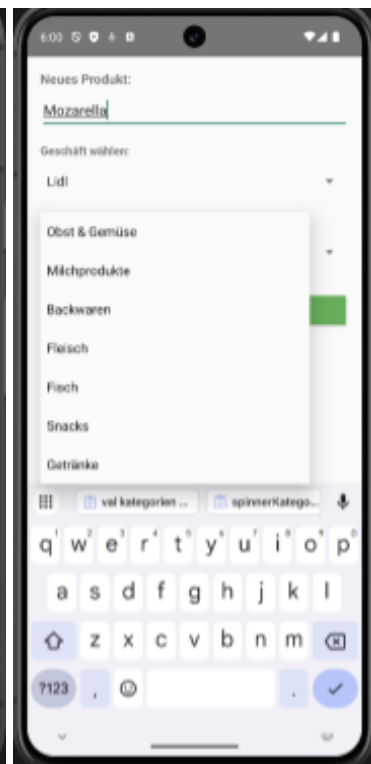
2.1



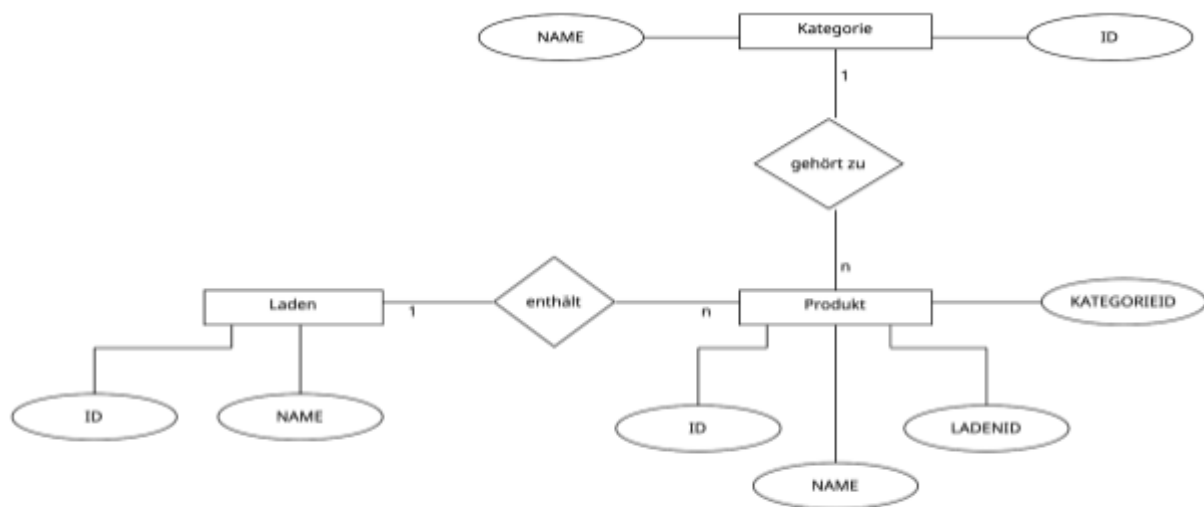
2.2



2.3



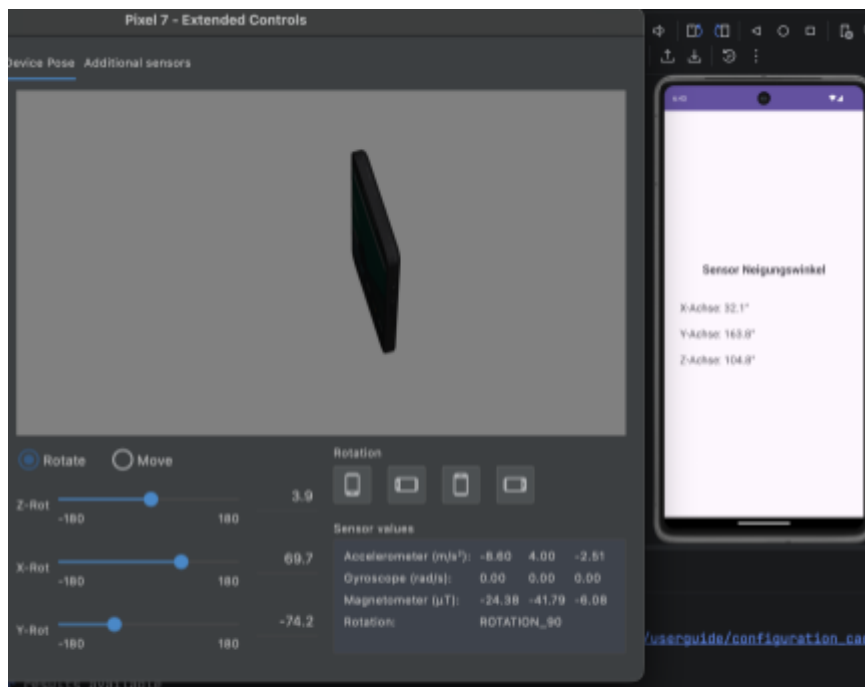
2.4



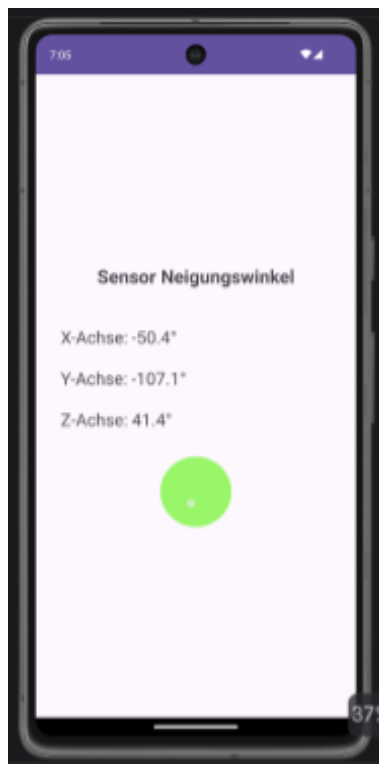
2.5



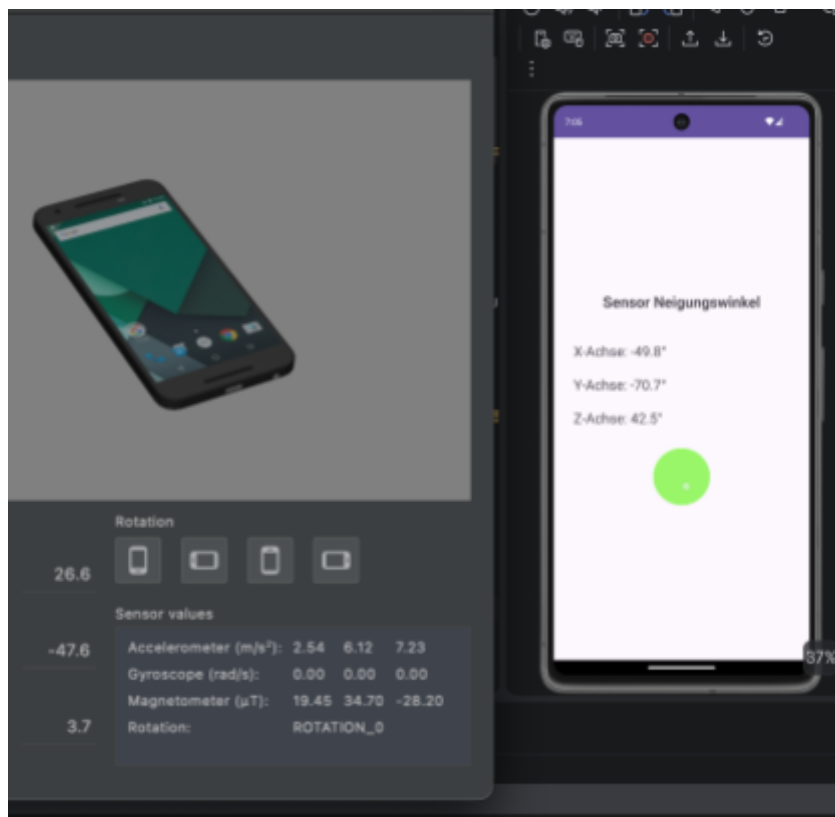
3.1



3.2



4.1



4.2